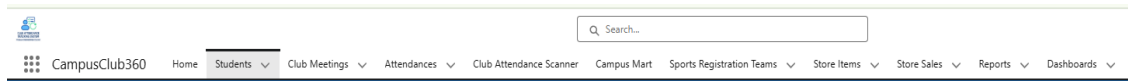


CampusClub360

Phase 6: User Interface Development

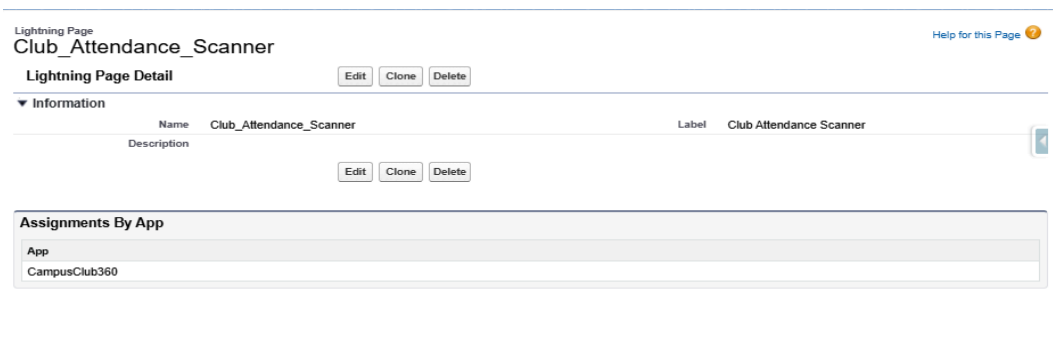
1. Custom App

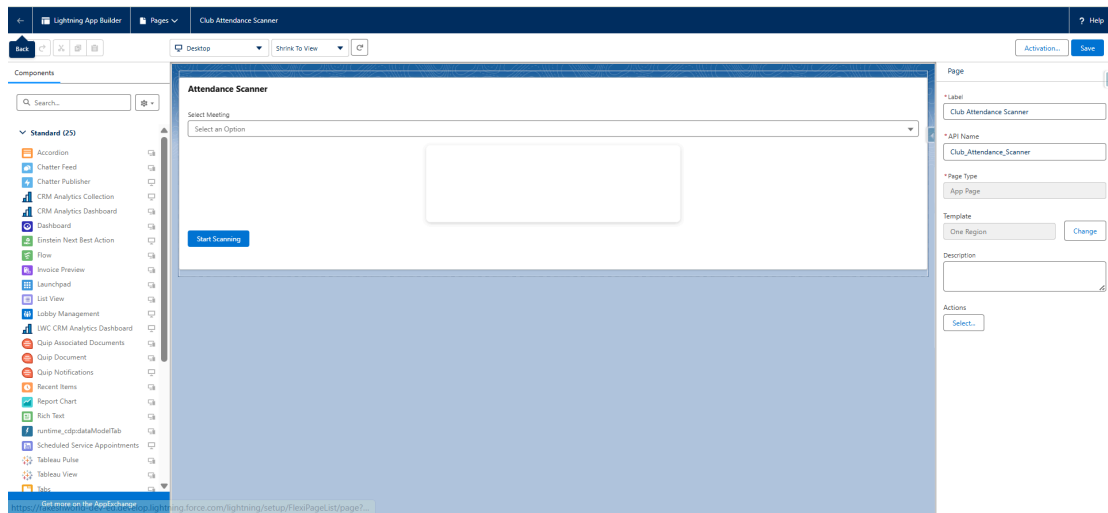
- **CampusClub360** → Central Salesforce app for managing students, clubs, sports, and store operations.
- **Tabs Included:**
 - Home
 - Students
 - Club Meetings
 - Attendance
 - Club Attendance Scanner
 - Campus Mart
 - Sports Registration Teams
 - Store Items
 - Store Sales
 - Reports
 - Dashboards



2. Lightning App Builder Pages

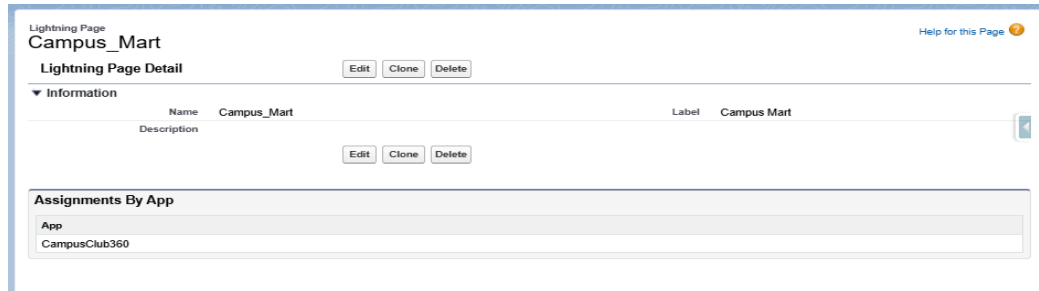
- **Club Attendance Scanner Page:**
 - **Purpose:** Allow faculty/club leaders to scan students' QR codes for attendance.
 - **Contains:** Custom LWC attendanceScanner.

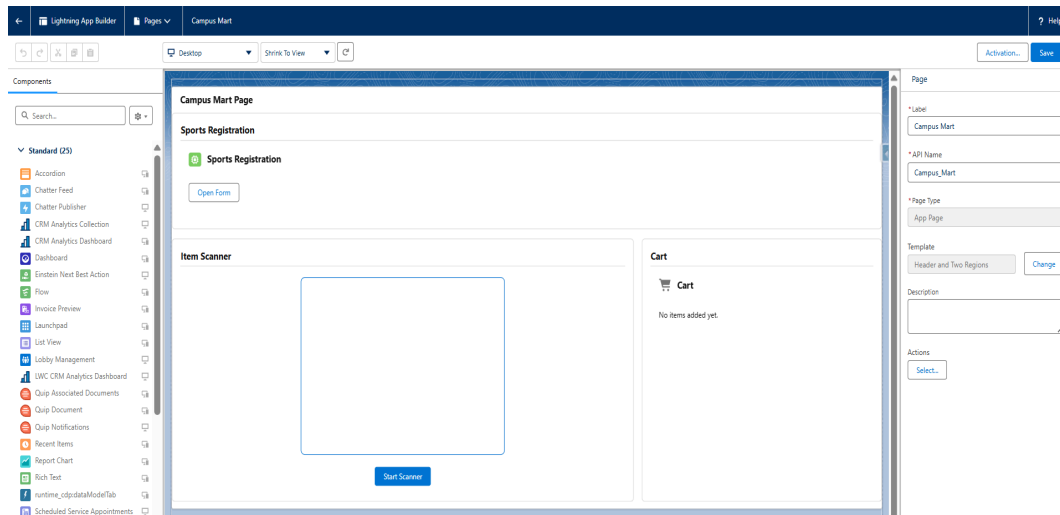




- **Campus Mart Page:**

- **Purpose:** Enable store salespersons to register sports teams and manage store sales.
- **Layout:**
 - Top: Sports Registration Form (via LWC).
 - Left: Item Scanner (via LWC).
 - Right: Cart (store sales integration).





3. Apex with LWC:

- **attendanceScanner:**
 - **Purpose:** Scans QR code of students for attendance.
 - **Logic:**
 - Prevents duplicate scan entries.
 - Stores attendance record in Attendance__c.
 - **Backend:** Integrated with AttendanceController.

attendanceScanner.html

```
<template>
  <lightning-card title="Attendance Scanner">
    <div class="slds-p-around_medium">

      <!-- Current meeting display -->
      <template if:true={selectedMeetingName}>
        <p class="meetingInfo"><strong>Current Meeting:</strong>
{selectedMeetingName}</p>
      </template>

      <!-- Meeting selection -->
      <lightning-combobox
        name="meeting"
        label="Select Meeting"
        options={meetingOptions}
        value={selectedMeeting}
        onchange={handleMeetingChange}>
      </lightning-combobox>

      <!-- Video feed -->
```

```

<div class="slds-m-top_medium scannerContainer">
  <video class="scanner-video" autoplay playsinline></video>
  <canvas class="scanner-canvas" style="display:none"></canvas>
</div>

<!-- Start/Stop button -->
<div class="slds-m-top_medium">
  <lightning-button
    variant="brand"
    label={buttonLabel}
    onclick={toggleScan}>
  </lightning-button>
</div>

<!-- Scanned results -->
<div class="slds-m-top_medium results">
  <template if:true={scannedList.length}>
    <h3>Recent Scans</h3>
    <template for:each={scannedList} for:item="s">
      <p key={s.key} class={s.cssClass}>
        {s.name} — {s.time}
      </p>
    </template>
  </template>
</div>

</div>
</lightning-card>
</template>

```

attendanceScanner.css

```

.success {
  color: #0a8f08; /* green */
  font-weight: 600;
}
.duplicate {
  color: #d9822b; /* orange */
  font-weight: 600;
}
.error {
  color: #c23934; /* red */
  font-weight: 600;
}

```

```

.meetingInfo {
  margin-bottom: 0.5rem;
}
.scannerContainer {
  display: flex;
  justify-content: center;
  align-items: center;
}
.scanner-video {
  width: 100%;
  max-width: 480px;
  border-radius: 8px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.15);
}

```

attendanceScanner.js

```

import { LightningElement, track } from 'lwc';
import { loadScript } from 'lightning/platformResourceLoader';
import { ShowToastEvent } from 'lightning/platformShowToastEvent';

import JSQR from '@salesforce/resourceUrl/JSQR';
import getMeetings from '@salesforce/apex/AttendanceController.getMeetings';
import findStudentByBarcode from
'@salesforce/apex/AttendanceController.findStudentByBarcode';
import createAttendance from
'@salesforce/apex/AttendanceController.createAttendance';

export default class AttendanceScanner extends LightningElement {
  @track meetingOptions = [];
  selectedMeeting = "";
  selectedMeetingName = "";
  isScanning = false;
  jsqrLoaded = false;
  stream = null;
  @track scannedList = [];

  get buttonLabel() {
    return this.isScanning ? 'Stop Scanning' : 'Start Scanning';
  }

  renderedCallback() {
    if (this.jsqrLoaded) return;

```

```

loadScript(this, JSQR)
  .then(() => {
    this.jsqrLoaded = true;
    return getMeetings();
  })
  .then(result => {
    // Map meetings; label shows name (you can add date if desired)
    this.meetingOptions = result.map(m => ({ label: m.Name, value: m.Id }));
  })
  .catch(error => {
    console.error('Error loading JSQR or fetching meetings', error);
    this.showToast('Load Error', 'Could not load scanner or meetings', 'error');
  });
}

handleMeetingChange(event) {
  this.selectedMeeting = event.detail.value;
  const meeting = this.meetingOptions.find(m => m.value ===
this.selectedMeeting);
  this.selectedMeetingName = meeting ? meeting.label : "";
}

async toggleScan() {
  if (!this.isScanning) {
    if (!this.selectedMeeting) {
      this.showToast('Select Meeting', 'Please select a meeting first.', 'warning');
      return;
    }
    await this.startCamera();
    this.isScanning = true;
    this.scanLoop();
  } else {
    this.stopCamera();
    this.isScanning = false;
  }
}

async startCamera() {
  const video = this.template.querySelector('video');
  try {
    this.stream = await navigator.mediaDevices.getUserMedia({ video: {
facingMode: 'environment' } });
    video.srcObject = this.stream;
  }
}

```

```

        await video.play();
    } catch (err) {
        console.error('Camera error', err);
        this.showToast('Camera Error', 'Cannot access camera. Allow permission or try
a different browser.', 'error');
    }
}

stopCamera() {
    if (this.stream) {
        this.stream.getTracks().forEach(t => t.stop());
        this.stream = null;
    }
}

disconnectedCallback() {
    // ensure camera is stopped when component removed
    this.stopCamera();
}

async scanLoop() {
    const video = this.template.querySelector('video');
    const canvas = this.template.querySelector('canvas');
    if (!canvas) return;
    const ctx = canvas.getContext('2d');

    const loop = async () => {
        if (!this.isScanning) return;
        if (video.readyState === video.HAVE_ENOUGH_DATA) {
            canvas.width = video.videoWidth;
            canvas.height = video.videoHeight;
            ctx.drawImage(video, 0, 0, canvas.width, canvas.height);
            const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);

            const code = window.jsQR ? window.jsQR(imageData.data, canvas.width,
canvas.height) : null;
            if (code && code.data) {
                await this.handleDetected(code.data);
                await this.sleep(1200); // debounce
            }
        }
        requestAnimationFrame(loop);
    };

```

```

        requestAnimationFrame(loop);
    }

    sleep(ms) {
        return new Promise(res => setTimeout(res, ms));
    }

    async handleDetected(barcodeValue) {
        try {
            const student = await findStudentByBarcode({ barcodeValue });
            if (!student) {
                this.showToast('Not Found', `No student found for code: ${barcodeValue}`,
                    'error');
                // show in recent list as error
                this.prependScan({ key: barcodeValue + '-' + Date.now(), name: `Unknown
                    (${barcodeValue})`, time: new Date().toLocaleString(), cssClass: 'error' });
                return;
            }

            // call server to create attendance (or return existing)
            const result = await createAttendance({ studentId: student.Id, meetingId:
                this.selectedMeeting });
            const att = result.attendance;
            const wasNew = result.isNew;

            if (wasNew) {
                this.showToast('Attendance Recorded', `${att.Student__r ?
                    att.Student__r.Name : student.Name} checked in successfully.`, 'success');
            } else {
                this.showToast('Duplicate', `${att.Student__r ? att.Student__r.Name :
                    student.Name} is already checked in for this meeting.`, 'warning');
            }

            this.prependScan({
                key: att.Id,
                name: att.Student__r ? att.Student__r.Name : student.Name,
                time: new Date(att.Check_In_Time__c).toLocaleString(),
                cssClass: wasNew ? 'success' : 'duplicate'
            });
        } catch (err) {
            console.error('Error processing scan', err);
        }
    }

```



```

        this.showToast('Error', 'An error occurred while processing the scan (or)
Attendance not allowed outside Club timings (10 AM – 3 PM, Mon–Sat, excluding
holidays', 'error');
    }
}

prependScan(scanObj) {
    this.scannedList = [scanObj, ...this.scannedList].slice(0, 10);
}

showToast(title, message, variant='info') {
    this.dispatchEvent(new ShowToastEvent({ title, message, variant }));
}
}

```

attendanceScanner.js-meta.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>58.0</apiVersion>
    <isExposed>true</isExposed>
    <targets>
        <target>lightning__AppPage</target>
        <target>lightning__HomePage</target>
        <target>lightning__RecordPage</target>
    </targets>
</LightningComponentBundle>

```

- **sportsRegistrationForm:**

- **Purpose:** Allows salespersons to register sports teams.
- **Fields:** Team Name, Number of Players, Sport, Registration Fee.
- **Logic:** On submit → record created in Sports_Registration__c.
- **Backend:** Integrated with SportsRegistrationController.

sportsRegistrationForm.html

```

<template>
    <lightning-card title="Sports Registration" icon-name="custom:custom63">
        <div class="slds-p-around_medium">
            <!-- Toggle Button -->
            <lightning-button label={toggleLabel} onclick={toggleForm}></lightning-button>
        </div>

        <!-- Registration Form -->
        <template if:true={isFormOpen}>

```

```

<div class="slds-p-around_medium">
  <lightning-combobox
    name="sport"
    label="Select Sport"
    value={sport}
    placeholder="Choose a sport"
    options={sportOptions}
    onchange={handleChange}>
  </lightning-combobox>

  <lightning-input
    name="teamName"
    label="Team Name"
    value={teamName}
    onchange={handleChange}>
  </lightning-input>

  <lightning-input
    name="players"
    label="Number of Players"
    type="number"
    value={players}
    onchange={handleChange}>
  </lightning-input>

  <lightning-input
    name="fee"
    label="Registration Fee"
    type="number"
    value={fee}
    onchange={handleChange}>
  </lightning-input>

  <div class="slds-m-top_medium">
    <lightning-button variant="brand" label="Register"
      onclick={handleRegister}></lightning-button>
  </div>
</div>
</template>
</lightning-card>
</template>

```

sportsRegistrationForm.js

```
import { LightningElement, track } from 'lwc';
import { ShowToastEvent } from 'lightning/platformShowToastEvent';
import createRegistration from
 '@salesforce/apex/SportsRegistrationController.createRegistration';

export default class SportsRegistrationForm extends LightningElement {
    @track isFormOpen = false;
    @track sport = "";
    @track teamName = "";
    @track players = 0;
    @track fee = 0;

    get toggleLabel() {
        return this.isFormOpen ? 'Close Form' : 'Open Form';
    }

    get sportOptions() {
        return [
            { label: 'Cricket', value: 'Cricket' },
            { label: 'Kabaddi', value: 'Kabaddi' },
            { label: 'Volley Ball', value: 'Volley Ball' },
            { label: 'Throw Ball', value: 'Throw Ball' },
            { label: 'Basket Ball', value: 'Basket Ball' },
            { label: 'Tennikoit', value: 'Tennikoit' },
            { label: 'Badminton', value: 'Badminton' },
            { label: 'Kho-Kho', value: 'Kho-Kho' },
            { label: 'Chess', value: 'Chess' },
            { label: 'Tug of War', value: 'Tug of War' },
            { label: 'Carroms', value: 'Carroms' },
            { label: 'Table-Tennis', value: 'Table-Tennis' },
            { label: 'Others', value: 'Others' }
        ];
    }

    toggleForm() {
        this.isFormOpen = !this.isFormOpen;
    }

    handleChange(event) {
        this[event.target.name] = event.target.value;
    }
}
```

```

handleRegister() {
  createRegistration({
    sport: this.sport,
    teamName: this.teamName,
    players: this.players,
    fee: this.fee
  })
  .then(() => {
    this.dispatchEvent(
      new ShowToastEvent({
        title: 'Success',
        message: 'Team Registered Successfully!',
        variant: 'success'
      })
    );
    this.resetForm();
  })
  .catch(error => {
    this.dispatchEvent(
      new ShowToastEvent({
        title: 'Error',
        message: error.body.message,
        variant: 'error'
      })
    );
  });
}

resetForm() {
  this.sport = "";
  this.teamName = "";
  this.players = 0;
  this.fee = 0;
}
}

```

sportsRegistrationForm.js-meta.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>64.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>

```

```
<target>lightning__AppPage</target>
<target>lightning__RecordPage</target>
<target>lightning__HomePage</target>
</targets>
</LightningComponentBundle>
```

- **sportsItemScanner (child component):**
 - **Purpose:** Scans barcodes of store items.
 - **Logic:**
 - Displays scanned item.
 - On "Add-To-Cart" → item moved to SportsItemList.
 - **Backend:** Integrated with SportsItemController.
-

sportsItemScanner.html

```
<template>
  <div class="scanner-container">
    <div class="video-box">
      <video></video>
    </div>

    <div class="scanner-footer">
      <lightning-button
        label={buttonLabel}
        onclick={toggleScanner}
        variant="brand">
      </lightning-button>
    </div>

    <!-- Show result -->
    <template if:true={scannedItem}>
      <div class="result-success">
        <p>{scannedItem.Name}</p>
        <p>Price: ₹{scannedItem.Price__c}</p>
        <lightning-button
          label="Add to Cart"
          onclick={addToCart}
          variant="success">
        </lightning-button>
      </div>
    </template>

    <template if:true={errorMsg}>
      <div class="result-error">{errorMsg}</div>
```

```
</template>
</div>
</template>
```

sportsItemScanner.css

```
.scanner-container {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  padding: 10px;
}

.video-box {
  width: 100%;
  max-width: 400px;
  height: 300px;
  border: 2px solid #0070d2; /* Salesforce blue */
  border-radius: 8px;
  overflow: hidden;
  margin-bottom: 10px;
}

video {
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.scanner-footer {
  margin-top: 10px;
  text-align: center;
}

.result-success {
  color: green;
  font-weight: bold;
  margin-top: 10px;
}

.result-error {
  color: red;
  font-weight: bold;
```

```
margin-top: 10px;
}
```

sportsItemScanner.js

```
import { LightningElement, track } from 'lwc';
import { loadScript } from 'lightning/platformResourceLoader';
import ZXingLib from '@salesforce/resourceUrl/ZXing';
import getItemByBarcode from
'@salesforce/apex/SportsItemController.getItemByBarcode';

export default class SportsItemScanner extends LightningElement {
  @track scannedItem; // Store scanned item details
  @track errorMsg;
  scannerRunning = false;
  zxingLoaded = false;
  codeReader;

  renderedCallback() {
    if (this.zxingLoaded) return;

    loadScript(this, ZXingLib)
      .then(() => {
        this.zxingLoaded = true;
        this.codeReader = new window.ZXing.BrowserBarcodeReader();
        console.log('✅ ZXing loaded successfully');
      })
      .catch(error => {
        this.errorMsg = 'Failed to load ZXing: ' + error;
        console.error(this.errorMsg);
      });
  }

  async startScanner() {
    try {
      const video = this.template.querySelector('video');
      const result = await this.codeReader.decodeOnceFromVideoDevice(undefined,
video);

      // Call Apex to get item details
      const item = await getItemByBarcode({ barcode: result.text });
      if (item) {
        this.scannedItem = item;
        this.errorMsg = null;
      }
    } catch (error) {
      console.error('Scanner error:', error);
    }
  }
}
```

```

        } else {
            this.scannedItem = null;
            this.errorMsg = ' ⚠️ Item not found in system';
        }

        this.stopScanner();
    } catch (err) {
        this.errorMsg = ' ⚠️ Scan failed: ' + err.message;
        console.error(err);
    }
}

stopScanner() {
    if (this.codeReader) {
        this.codeReader.reset();
    }
    this.scannerRunning = false;
}

toggleScanner() {
    if (this.scannerRunning) {
        this.stopScanner();
    } else {
        this.startScanner();
        this.scannerRunning = true;
    }
}

get buttonLabel() {
    return this.scannerRunning ? 'Stop Scanner' : 'Start Scanner';
}

// Add item to cart
// sportsItemScanner.js
addToCart() {
    if (this.scannedItem) {
        this.dispatchEvent(new CustomEvent('addtocart', {
            detail: {
                id: this.scannedItem.Id, // very important
                name: this.scannedItem.Name,
                price: Number(this.scannedItem.Price__c),
                barcode: this.scannedItem.Barcode__c,
                quantity: 1
            }
        }));
    }
}

```



```

    }
    });
    this.scannedItem = null;
  }
}
}

```

sportsItemScanner.js-meta.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>64.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightning__AppPage</target>
    <target>lightning__RecordPage</target>
    <target>lightning__HomePage</target>
  </targets>
</LightningComponentBundle>

```

- **sportsItemList (child component):**

- **Purpose:** Displays all items added from the scanner.
 - **Logic:**
 - **Backend:** Integrated with SportsSaleController.
-

sportsItemList.html

```

<template>
  <lightning-card title="Cart" icon-name="utility:cart">
    <template if:true={cartItems.length}>
      <template for:each={cartItems} for:item="item">
        <div key={item.id} class="cart-item">
          <span class="item-name">{item.name}</span>
          <span class="item-price">
            ₹{item.price} × {item.quantity} = ₹{item.lineTotal}
          </span>
          <lightning-button-icon
            icon-name="utility:delete"
            variant="destructive"
            alternative-text="Remove"
            title="Remove"
            onclick={removeItem}
            data-id={item.id}>
          </lightning-button-icon>

```

```

        </div>
    </template>
    <div class="total">Total: ₹{totalAmount}</div>
    <lightning-button
        label="Done"
        variant="brand"
        onclick={finalizeSale}>
    </lightning-button>
</template>
<template if:false={cartItems.length}>
    <p class="slds-p-around_medium">No items added yet.</p>
</template>

<!--Custom Toast inside component -->
<template if:true={toast.visible}>
    <div class={toastClass}>
        {toast.message}
    </div>
</template>
</lightning-card>
</template>

```

sportsItemList.css

```

.cart-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin: 0.3rem 0;
}

.item-name {
    font-weight: bold;
}

.item-price {
    margin-left: 1rem;
    flex-grow: 1;
}

.total {
    font-weight: bold;
    margin-top: 1rem;
    text-align: right;
}

```

```
}

/*Custom Toast Styles */
.success {
  background: #2ecc71;
  color: white;
  padding: 0.5rem;
  margin-top: 1rem;
  border-radius: 0.25rem;
}
```

```
.error {
  background: #e74c3c;
  color: white;
  padding: 0.5rem;
  margin-top: 1rem;
  border-radius: 0.25rem;
}
```

sportsItemList.js

```
import { LightningElement, track, api } from 'lwc';
import finalizeSale from '@salesforce/apex/SportsSaleController.finalizeSale';
```

```
export default class SportsItemList extends LightningElement {
  @track cartItems = [];
  @track totalAmount = 0;
```

```
  // toast state
```

```
  @track toast = { visible: false, message: "", type: "" };
```

```
  showToast(message, type = 'info') {
    this.toast = { visible: true, message, type };
```

```
    // auto hide after 3 sec
```

```
    setTimeout(() => {
      this.toast.visible = false;
    }, 3000);
```

```
  }
```

```
  // Add item
```

```
  @api addItem(item) {
    let existing = this.cartItems.find(i => i.id === item.id);
    if (existing) {
```

```

        existing.quantity += 1;
        existing.lineTotal = existing.price * existing.quantity;
    } else {
        this.cartItems = [
            ...this.cartItems,
            { ...item, quantity: 1, lineTotal: item.price }
        ];
    }
    this.calculateTotal();
}

removeItem(event) {
    const id = event.currentTarget.dataset.id;
    let index = this.cartItems.findIndex(i => i.id === id);
    if (index > -1) {
        if (this.cartItems[index].quantity > 1) {
            this.cartItems[index].quantity -= 1;
            this.cartItems[index].lineTotal =
                this.cartItems[index].price * this.cartItems[index].quantity;
        } else {
            this.cartItems.splice(index, 1);
        }
        this.cartItems = [...this.cartItems];
    }
    this.calculateTotal();
}

calculateTotal() {
    this.totalAmount = this.cartItems.reduce((sum, i) => sum + i.lineTotal, 0);
}

finalizeSale() {
    finalizeSale({
        itemsJson: JSON.stringify(this.cartItems),
        totalAmount: this.totalAmount
    })
    .then(() => {
        this.showToast('✅ Sale finalized successfully!', 'success');
        this.cartItems = [];
        this.totalAmount = 0;
    })
    .catch((err) => {
        let message = 'Unknown error';

```

```

        if (err && err.body && err.body.message) {
            message = err.body.message;
        } else if (err && err.message) {
            message = err.message;
        }
        this.showToast('❌ Failed to finalize sale: ' + message, 'error');
    });
}
}

```

sportsItemList.js-meta.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>64.0</apiVersion>
    <isExposed>true</isExposed>
    <targets>
        <target>lightning__RecordPage</target>
        <target>lightning__AppPage</target>
        <target>lightning__HomePage</target>
    </targets>
</LightningComponentBundle>

```

4. LWC (Lightning Web Components):

- **sportsClubPage (parent component):**
 - **Purpose:** Acts as the main container that connects:
 - **Top:** Sports Registration Form
 - **Left:** Sports Item Scanner
 - **Right:** Sports Item List
 - **Logic:** Coordinates data flow → scanner → list → sales record.

sportsClubPage.html

```

<template>
    <lightning-card title="Campus Mart Page">
        <div class="sports-club-container">

            <!-- Top: Sports Registration -->
            <div class="section registration">
                <h2>Sports Registration</h2>
                <c-sports-registration-form></c-sports-registration-form>
            </div>

            <!-- Bottom split: Scanner and Cart -->
            <div class="content">

```

```

        <div class="section scanner">
            <h2>Item Scanner</h2>
            <c-sports-item-scanner
                onaddtocart={handleAddToCart}>
            </c-sports-item-scanner>
        </div>

        <div class="section cart">
            <h2>Cart</h2>
            <c-sports-item-list
                onfinalize={handleFinalize}>
            </c-sports-item-list>
        </div>
    </div>
</lightning-card>
</template>

```

sportsClubPage.css

```

.sports-club-container {
    display: flex;
    flex-direction: column;
    gap: 1rem;
}

.section {
    border: 1px solid #dcdcdc; /* light grey border */
    border-radius: 6px;
    padding: 1rem;
    background: #ffffff;
}

.section h2 {
    font-size: 1rem;
    font-weight: bold;
    margin-bottom: 0.5rem;
    border-bottom: 1px solid #ddd;
    padding-bottom: 0.3rem;
}

.content {
    display: flex;
    gap: 1rem;
}

```

```
}  
  
.scanner {  
  flex: 2;  
}
```

```
.cart {  
  flex: 1;  
}
```

sportsClubPage.js

```
import { LightningElement } from 'lwc';
```

```
export default class SportsClubPage extends LightningElement {
```

```
  // Handle add-to-cart from scanner  
  handleAddToCart(event) {  
    const cart = this.template.querySelector('c-sports-item-list');  
    if (cart) {  
      cart.addItem(event.detail);  
    }  
  }  
}
```

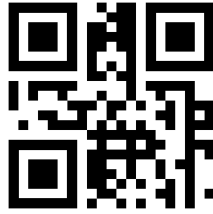
sportsClubPage.js-meta.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">  
  <apiVersion>64.0</apiVersion>  
  <isExposed>true</isExposed>  
  <targets>  
    <target>lightning__RecordPage</target>  
    <target>lightning__AppPage</target>  
    <target>lightning__HomePage</target>  
  </targets>  
</LightningComponentBundle>
```

5. Student QR Codes:



Student 1



Student 2



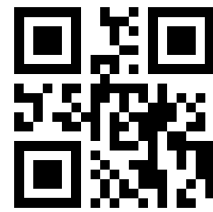
Student 3



Student 4



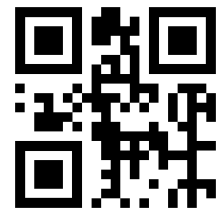
Student 5



Student 6



Student 7



Student 8



Student 9



Student 10

6.Sports Item Barcodes:



BALL001

Cricket Ball 1



BALL002

Cricket Ball 2



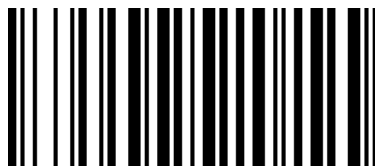
BAT001

Cricket Bat 1



BAT002

Cricket Bat 2



BK002

Chemistry Textbook



BG001

Backpack (College)